# Retail Sales Analysis

## Trainer Guide | Python Data Analysis Graduation Project | 2 Hours

Dataset note: all records are synthetic. Use results as classroom examples, not claims about a real retailer.

### 1. Business scenario

A household-goods retailer runs three branches and sells products across four categories. Management wants to know which categories and products contribute the most sales and profit, whether discounts are eroding margins, and how branches differ. Students will turn raw transactions into a cleaned dataset, analysis, visualizations, and a simple interactive dashboard.

### 2. Learning outcomes

• Import a CSV and inspect dimensions, data types, missing values, duplicates, and distributions.

• Use NumPy percentiles and median; use Pandas for cleaning, derived columns, grouping, and KPI calculations.

• Use Matplotlib and Seaborn for static charts and diagnostics, then Plotly for interactive charts.

• Build a Streamlit dashboard with filters, KPI cards, charts, and data-quality notes.

• Make evidence-based commercial recommendations about products, discounts, and branches.

### 3. Two-hour lesson plan

| Time | Trainer action | Student output |
|---|---|---|
| 0–10 min | Introduce business questions and define sales, profit, margin, average transaction. | List 3 business questions. |
| 10–25 min | Read CSV; inspect shape, head, info, missing values, duplicates. | Data-quality checklist. |
| 25–45 min | Convert dates; remove exact duplicates; validate prices, quantities, discounts, and returns. | Cleaned dataset; discuss suspicious sales. |
| 45–70 min | Create month and profit margin; group by category, product, branch, and payment method. | KPI table and 3 insights. |
| 70–90 min | Build Matplotlib, Seaborn, and Plotly charts. | At least 3 useful charts. |
| 90–115 min | Run the Streamlit dashboard and demonstrate filters and KPI cards. | Interactive dashboard. |
| 115–120 min | Each group presents one evidence-backed recommendation. | One finding + one action. |

### 4. KPI definitions

| KPI | Definition | Interpretation |
|---|---|---|
| Total sales | Sum of transaction Sales_EGP. | Revenue after discounts. |
| Total profit | Sum of Sales_EGP minus estimated Cost_EGP. | Check how costs are represented before using in a real business. |
| Profit margin | Total profit ÷ total sales × 100. | Prefer aggregate profit / aggregate sales, not a simple average of row ma |
| Average transaction | Total sales ÷ number of transactions (or mean transaction value if each row is one transaction). | In this dataset, each row represents one transaction. |
| Discount rate | Discount percentage recorded on each transaction. | High discount is not automatically bad; compare it with profit and volume |

### 5. Guided code sequence

#### *Import and inspect*

```
raw = pd.read_csv("retail_sales_raw.csv")
print(raw.shape)
print(raw.head())
print(raw.info())
print(raw.isna().sum())
print("Duplicates:", raw.duplicated().sum())
```

NumPy: use the median and percentiles to understand skew and extreme values.

```
sales = pd.to_numeric(raw["Sales_EGP"], errors="coerce").dropna().to_numpy()
print("Median:", np.median(sales))
print("95th percentile:", np.percentile(sales, 95))
```

#### *Cleaning and feature engineering*

```
df = raw.copy()
df["Order_Date"] = pd.to_datetime(df["Order_Date"], errors="coerce")
df = df.drop_duplicates()
df.loc[(df["Discount_Rate"] < 0) | (df["Discount_Rate"] > .8), "Discount_Rate"] = np.nan
```

```
df.loc[df["Unit_Price_EGP"] <= 0, "Unit_Price_EGP"] = np.nan
df.loc[df["Quantity"] <= 0, "Quantity"] = np.nan
df["Payment_Method"] = df["Payment_Method"].fillna("Unknown")
df["Discount_Rate"] = df["Discount_Rate"].fillna(df["Discount_Rate"].median())
df["Unit_Price_EGP"] = df["Unit_Price_EGP"].fillna(
    df.groupby("Category")["Unit_Price_EGP"].transform("median"))
df["Quantity"] = df["Quantity"].fillna(df["Quantity"].median())
df["Sales_EGP"] = (df["Unit_Price_EGP"] * df["Quantity"] * (1-df["Discount_Rate"])).round(2)
df["Profit_EGP"] = (df["Sales_EGP"] - df["Cost_EGP"]).round(2)
df["Order_Month"] = df["Order_Date"].dt.to_period("M").astype(str)
df["Profit_Margin"] = np.where(df["Sales_EGP"] > 0,
    df["Profit_EGP"] / df["Sales_EGP"], np.nan)
```

## 6. Visualization choices

• Matplotlib: horizontal bar chart comparing total sales by category.

• Seaborn: boxplot of profit margin by category to inspect spread and unusual values.

• Plotly: interactive monthly sales line chart or branch sales-versus-profit scatter plot.

## 7. Dashboard walkthrough

• Install packages: pip install -r requirements.txt

• Run the app: streamlit run app.py

• Demonstrate branch, category, payment method, and customer type filters.

• Explain that KPIs and charts recalculate from the filtered data.

• Open Data quality checks and Explore filtered data; show the CSV download.

## 8. Discussion prompts and answer cues

**Which category is best?** Compare total profit and margin, not sales alone; check volume and transaction count.

**Should we stop discounting?** Not automatically. Compare discount levels with profit, units sold, and sales before/after a promotion if such data is available.

**Which branch performs best?** Compare sales, profit, and transaction count; a high-revenue branch may also have higher operating costs, which are not included here.

**What should happen to outliers?** Validate suspicious prices and sales against receipts. Capping is useful for sensitivity analysis, not a substitute for source verification.

**What makes a good recommendation?** It names the action, evidence, expected benefit, and a metric to monitor.

## 9. Final deliverable checklist

■ Cleaned dataframe and documented data-quality decisions.

■ At least 3 KPIs and 3 useful visualizations.

■ A Streamlit dashboard with filters and clear labels.

■ Three findings and two actionable recommendations linked to metrics.

Instructor note: keep the session guided. The Streamlit app is provided as a working reference so class time can focus on data reasoning, code comprehension, and the link between findings and business actions.